



**Department of Electrical, Computer
& Biomedical Engineering**
Faculty of Engineering & Architectural Science

Programs: Electrical & Computer Engineering

Course Number	COE758
Course Title	Digital Systems Engineering
Semester/Year	Fall 2025
Instructor	Dr. Lev Kirischian

Project Report No.	2
---------------------------	----------

Report Title	Simple Video Game Processor for VGA
--------------	-------------------------------------

Section No.	3
Submission Date	Nov 17, 2025
Due Date	Nov 17, 2025

Name	Student ID	Signature*
Bilal Irfan	****76502	
Umair Irfan	****90966	U.I

(Note: remove the first 4 digits from your student ID)

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at:*

<https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

Table of Contents

Abstract	3
Introduction	4
System Specifications	5
VGA Controller Specification	5
Static Display Elements	6
Dynamic Game Elements	6
Game Behavior and Logic	6
Device Design	7
Symbol	8
VGA Specifications	8
Block Diagram	9
State Diagram	10
Process Diagram	11
Results	12
Conclusion	14
Appendix	15
Source Code	15
vga_timing.vhd	15
refresh_controller.vhd	18
field_renderer.vhd	18
player_movement.vhd	21
ball_physics.vhd	24
pong_top.vhd	27
pong.ucf	32

Abstract

This report presents the design and implementation of a Simple Video Game Processor (SVGP) for VGA display, developed as part of the COE758 Digital Systems Engineering course. The project demonstrates real-time signal generation and FPGA-to-I/O device interfacing using a Xilinx Spartan-3E FPGA platform programmed with VHDL in the Xilinx ISE environment. The SVGP implements a functional video game similar to classic Pong, featuring both static and dynamic visual elements rendered at 640×480 resolution with a 60 Hz refresh rate. The static game field consists of a green playing area bordered by white boundaries, while dynamic elements include a yellow ball and two player paddles (blue and pink). The system utilizes precise VGA timing parameters, with horizontal synchronization operating at 800 clock cycles per line and vertical synchronization at 525 lines per frame, driven by a 25 MHz pixel clock.

The game logic incorporates collision detection algorithms that modify the ball's trajectory by 90 degrees upon impact with borders or player paddles. When the ball passes through goal areas on either side, it temporarily changes to red before respawning at the center in yellow. Player control is achieved through FPGA board switches, which are mapped to vertical paddle movement. The design uses a VHDL architecture consisting of separate components for VGA timing control, RGB signal generation, player movement logic, ball physics, and collision detection. The VGA controller manages horizontal and vertical sync signals (HSYNC and VSYNC) while coordinating real-time pixel rendering across three 8-bit color channels. This approach ensures maintainable code and allows debugging during the hardware integration phase.

The project successfully demonstrates the synthesis of digital logic design, real-time signal processing, and human-computer interaction within a single embedded system. Through this implementation, practical experience was gained in VGA interface specifications, FPGA resource utilization, and the challenges of developing time-critical applications on reconfigurable hardware platforms.

Introduction

VGA technology is an analog standard connector used for computer video input, defining both the hardware interface and analog signaling for transferring graphical data to the display. VGA utilizes a 15-pin connector, with pins for crucial signals such as horizontal and vertical sync, RGB, and ground, all signals that are emulated in this project, as illustrated in Figure 1. These signals collectively enable precise timing and data transmission, ensuring the accurate rendering of graphical information on the display.

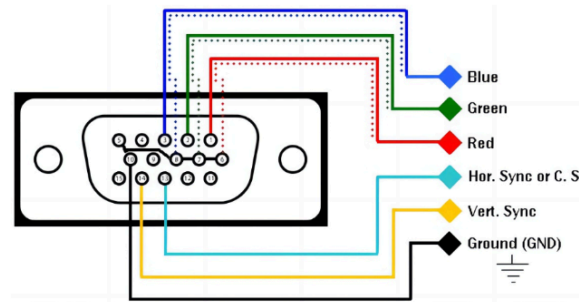


Figure 1. VGA Connector Pinout

VGA uses analog signals for video output and relies on two synchronization signals, horizontal and vertical sync, to properly position and display graphical data. The system "prints" pixels row-by-row, coordinated by the pixel clock, with the sync signals ensuring each row is completed before moving to the next. Horizontal and vertical pixel position is tracked based on the movement of pixel addressing. As shown in Figure 2, the display screen is treated as an $M \times N$ matrix, where a single frame is completed when all pixels in the matrix are addressed. The process starts at the top-left corner of the screen, moving horizontally across the row. Once the last pixel in the row is reached, the horizontal position is reset to the leftmost pixel, and the vertical position is incremented by one row. This cycle repeats until the bottom of the screen is reached.

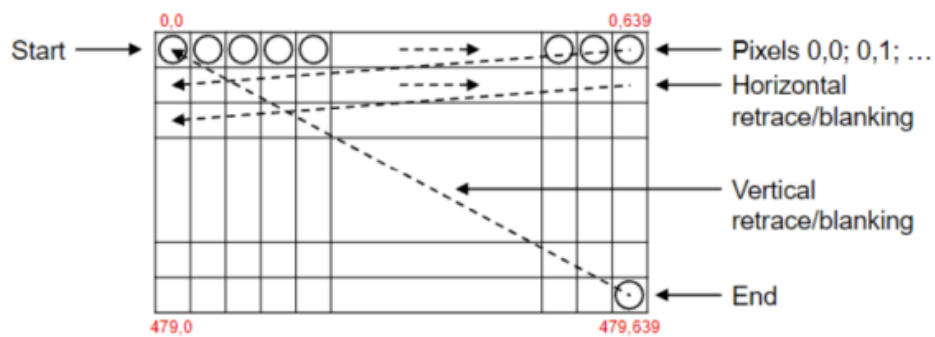


Figure 2. Pixel "movement" for one frame of an image

VGA, like most graphical display systems, uses RGB data to represent colors on the monitor. The RGB values are transmitted from the VGA controller as binary data (8 bits per channel in this project) and converted into analog voltage levels by digital-to-analog converters (DACs). These analog signals, ranging from 0 to approximately 0.7V, drive the intensity of red, green, and blue channels, allowing the display to render 16.7 million possible color combinations with smooth transitions and accurate timing. Colors are managed using three separate 8-bit arrays corresponding to the red, green, and blue signals, which are then converted to analog by interfacing with the Spartan-3E board.

The video game implemented in this project is a simplified version of the classic "Pong" game. The game features three dynamic elements: two player-controlled paddles (a blue paddle on the left and a pink paddle on the right) that can be controlled using onboard switches. Moving the switches upward moves the paddle up, while moving them downward shifts the paddle down. The third dynamic element is a yellow ball that travels across the screen, changing its direction upon interacting with different areas of the display. When the ball enters a goal on either the left or right side of the screen, it turns red and disappears, only to reappear at the center of the screen. The x and y pixel positions of the paddles and the ball are continuously tracked to determine interactions, such as collisions between the paddles, the white border, and the ball. Dynamic elements like the paddles and ball are rendered using their coordinates in conjunction with horizontal and vertical pixel trackers. By integrating these principles, this project demonstrates the functionality and practical implementation of a VGA controller on a Xilinx Spartan-3E FPGA platform, creating a Simple Video Game Processor (SVGP) that implements both static elements (a green playing field with white borders) and dynamic game components while operating in real-time.

System Specifications

The SVGP and VGA controllers implemented in Xilinx ISE are designed to operate on a Xilinx Spartan-3E FPGA platform, interfacing with a VGA monitor to create a functional video game processor. The system's core specifications include hardware timing requirements, signal generation, and game functionality.

VGA Controller Specification

The VGA controller manages real-time display output at 640×480 resolution with a 60 Hz refresh rate, driven by a 25 MHz pixel clock generated from the 50 MHz FPGA core clock. This pixel clock ensures each pixel is displayed at precisely 40-nanosecond intervals, maintaining synchronization across the entire display. The controller generates three primary outputs: 8-bit RGB signals representing pixel color data, and HSync/VSync synchronization signals that coordinate the electron beam positioning to "print" pixels row-by-row. Horizontal timing parameters define 800 total clock cycles per line, consisting of 640 active pixels, a 16-cycle front

porch, a 96-cycle sync pulse, and a 48-cycle back porch. Vertical timing parameters specify 525 total lines per frame, including 480 active lines, a 10-line front porch, a 2-line sync pulse, and a 33-line back porch. These parameters combine to produce 420,000 clock cycles per frame, resulting in a complete frame time of 16.8 milliseconds.

Static Display Elements

The system generates a static video frame displaying the game field background and boundaries. The playing field consists of a green background area that fills the active display region, bordered by white walls on all four sides. These static elements remain constant throughout gameplay and serve as the foundation upon which dynamic game elements are rendered. The pixel positioning system tracks horizontal and vertical coordinates to determine when border pixels should be displayed versus background pixels, using the VGA timing counters as reference points.

Dynamic Game Elements

Three dynamic elements comprise the interactive gameplay components: a yellow ball and two player-controlled paddles. The ball moves continuously across the playing field, changing trajectory upon collision with borders or paddles. Player one controls a blue paddle on the left side of the screen, while player two controls a pink paddle on the right side. Both paddles move exclusively in the vertical direction, positioned in front of goal areas on their respective sides. Moving the switches upward translates to upward paddle movement, while downward switch positions shift the paddles down. The x and y pixel positions of all dynamic elements are continuously tracked to enable real-time collision detection and interaction logic.

Game Behavior and Logic

The system implements collision detection algorithms that control ball trajectory and scoring mechanics. When the ball collides with the top or bottom borders or upon contact with a player's paddle, the ball's trajectory changes by 90 degrees relative to the point of impact, reversing its horizontal direction and sending it toward the opposing player. If a paddle fails to intercept the ball, allowing it to pass through the goal area on either the left or right side, the ball changes color from yellow to red, indicating a point has been scored. The red ball then disappears after briefly displaying, followed by respawning at the center of the screen in its original yellow color to resume gameplay (it is programmed to move in a direction opposite to its previous movement for fair gameplay). This behavior cycle continues throughout the game, with player inputs interpreted in real-time to control paddle positions and defend against the moving ball.

Table 1. Dimensions & Properties of Game Objects

Object	Size	Color	Position
Playing Field	580 x 400 pixels	Black (RGB: 0, 0, 0)	Centered with 30-pixel inset
White Borders	10 pixels	White (RGB: 255, 255, 255)	Around playing field
Left Paddle (P1)	16 x 90 pixels	Green (RGB: 0, 255, 0)	X = 48 pixels from left
Right Paddle (P2)	16 x 90 pixels	Cyan (RGB: 0, 255, 255)	X = 576 pixels from left
Ball	12 pixel diameter	Yellow/Red	Starts at center (320, 240)
Goal Openings	160 pixels high	Black	Y = 165 to 325 (left & right)

Device Design

The SVGP Xilinx project was successfully implemented and compiled on the Xilinx ISE CAD platform, displaying the Pong video game on a VGA monitor. The design consists primarily of VHDL files implementing the functionality and processes of the VGA controller, along with a constraints file provided in the project manual. The VHDL files manage the core operations of the VGA controller by generating a 25 MHz pixel clock from the 50 MHz core clock of the Spartan-3E FPGA. This pixel clock determines the rate at which pixel data is transmitted to the display. Horizontal and vertical sync pulses coordinate the transition of the electron beam between rows and columns, remaining high during active display and going low during the transition between front porch and back porch. The RGB color values displayed on the monitor are determined by the current position of the electron beam within the display area. For example, when the beam is near the screen edges, white pixels are rendered to create the game borders. These conditional color assignments are handled through numerous conditional statements based on the beam's current position, with separate logic for rendering static elements (borders and background) and dynamic elements (paddles and ball). This process makes up the functionality of the VHDL implementation, effectively emulating the behavior of a VGA controller to create the SVGP.

Symbol

The symbol diagram, shown in Figure 3, represents the top-level entity of the VGA controller, illustrating all input and output signals. The left side displays the input signals, including the 50 MHz clock signal (clk) from the FPGA board and four switch inputs (Switches) for user control. These four switches map to the onboard switches SW0 through SW3, where SW0 functions as a reset signal, SW1 controls the right paddle's vertical movement, SW2 may serve as a pause function or remain unused, and SW3 controls the left paddle's vertical movement. The right side shows six output signals generated by the VHDL program. The first three outputs are 8-bit buses labeled R_out, G_out, and B_out, which carry the red, green, and blue color channel data for each pixel. These 8-bit digital values provide 256 intensity levels per color channel, allowing for 16.7 million possible color combinations. The remaining three single-bit outputs are H_sync (horizontal synchronization), V_sync (vertical synchronization), and DAC_clk (the 25 MHz pixel clock). The H_sync and V_sync signals notify the VGA monitor when to begin drawing new lines or frames, ensuring proper pixel positioning. The DAC_clk output provides the 25 MHz timing signal required for synchronizing the digital-to-analog conversion process and pixel rendering. The VGA monitor receives all these output signals at the 25 MHz rate (every 40 nanoseconds) to draw and continuously update the game interface.

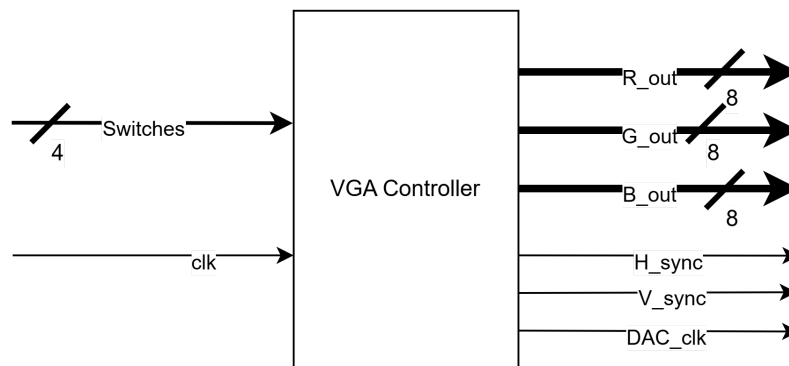


Figure 3. VGA Controller Symbol

VGA Specifications

All horizontal time periods are specified in multiples of the VGA pixel clock, which is 25 MHz with a 60 Hz refresh rate display. All vertical (frame) time periods are specified in multiples of VGA lines.

Table 2. Horizontal Frame Specifications

Parameter	Clock Cycles
Complete Line	800
Front Porch	16
Sync Pulse	96
Back Porch	48
Active Image Area	640

Table 3. Vertical Frame Specifications

Parameter	Lines
Complete Frame	525
Front Porch	10
Sync Pulse	2
Back Porch	3
Active Image Area	480

Block Diagram

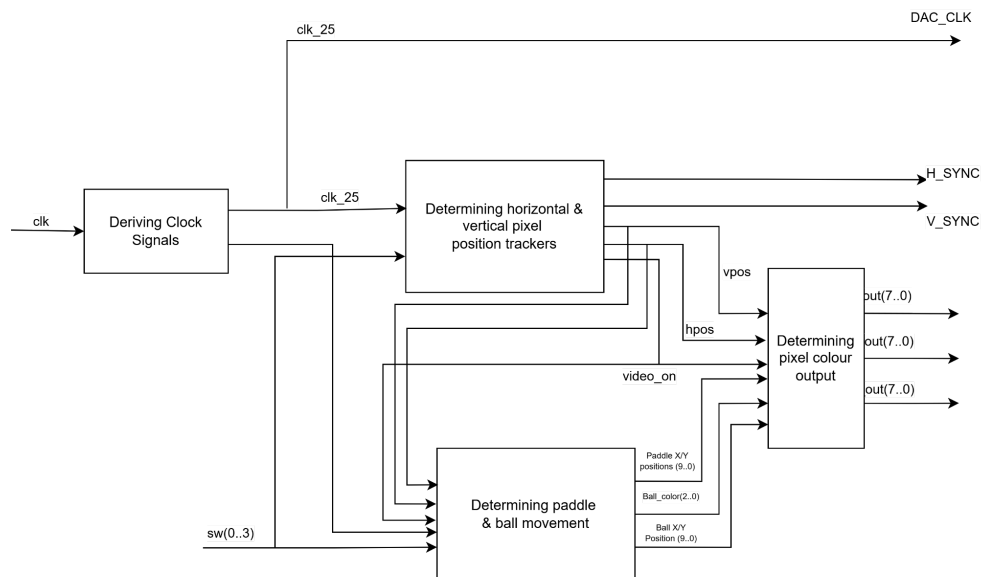


Figure 4: System block diagram showing module interconnections and signal flow

The block diagram, illustrated in Figure 4, provides a detailed architectural view of the internal processing flow within the video game processor. The design begins with clock generation on the left side, where the 50 MHz FPGA core clock (clk) is processed by the "Deriving Clock Signals" module to produce two critical timing signals: the 25 MHz pixel clock (clk25) used for VGA timing compliance, and a slower movement clock used to update the positions of dynamic game elements at a human-perceivable rate. The original clock speeds of 50 MHz or 25 MHz are far too fast for updating paddle and ball positions; at these speeds, updates would occur so rapidly that smooth motion would not be visible to players. The 25 MHz clock feeds into the "Determining horizontal and vertical pixel position trackers" module, which maintains horizontal (hpos) and vertical (vpos) position counters that track the current pixel being drawn on the screen. This module also generates the three primary synchronization outputs: DAC_clk (25 MHz pixel clock), H (horizontal sync), and V (vertical sync) signals, which are sent directly to the VGA output. The switch inputs (SW[0..3]) and clock are processed by the "Determining paddle and ball movement/speed" module, which implements the game control logic. This module updates the x and y coordinates of both paddles (Left/Right paddle X/Y positions) and the ball (Ball X/Y position, Ball direction) based on player inputs and the current game state. The collision detection algorithms within this module determine when the ball interacts with paddles, borders, or goal areas, updating the ball's trajectory accordingly. Finally, the "Determining pixel colour output" module receives the current pixel position (hpos, vpos), the bright signal, and all game element coordinates to determine which color should be displayed at each pixel location. This module outputs three 8-bit color channel values: R_out[7..0], G_out[7..0], and B_out[7..0], which are then sent to the Spartan-3E board's DAC for conversion to analog signals and display on the VGA monitor. The design architecture ensures that timing-critical operations (pixel tracking and sync generation) remain independent from game logic, allowing for smooth, real-time rendering at the 60 Hz refresh rate.

State Diagram

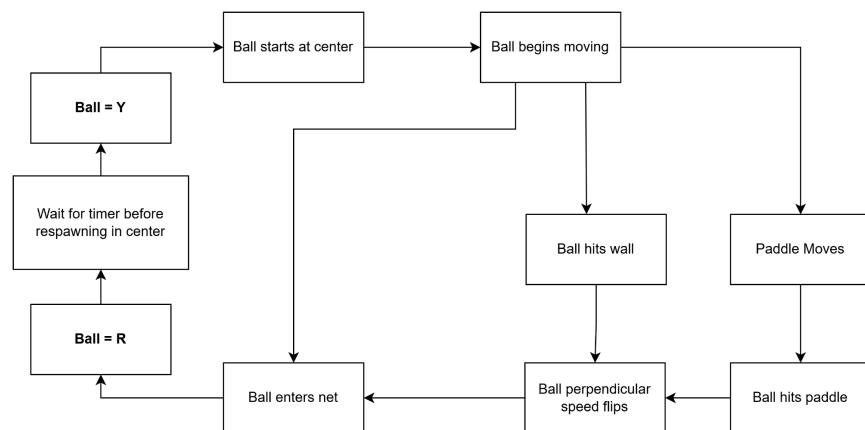


Figure 5: State Diagram of Gameplay

The state diagram in Figure 5 illustrates the state and transition characteristics of the ball's behavior in terms of in-game events. The diagram includes nine total states and their corresponding conditions for transitioning between them. Each state models the physical movement of the ball, the collision response of the ball interacting with other elements of the game (walls, paddles, net), or how the ball is scored/reset. Although many of these behaviors may occur very rapidly during actual real-time operation, all of the states are included to show how the hardware-based behavior of the ball is modeled in VHDL.

Process Diagram

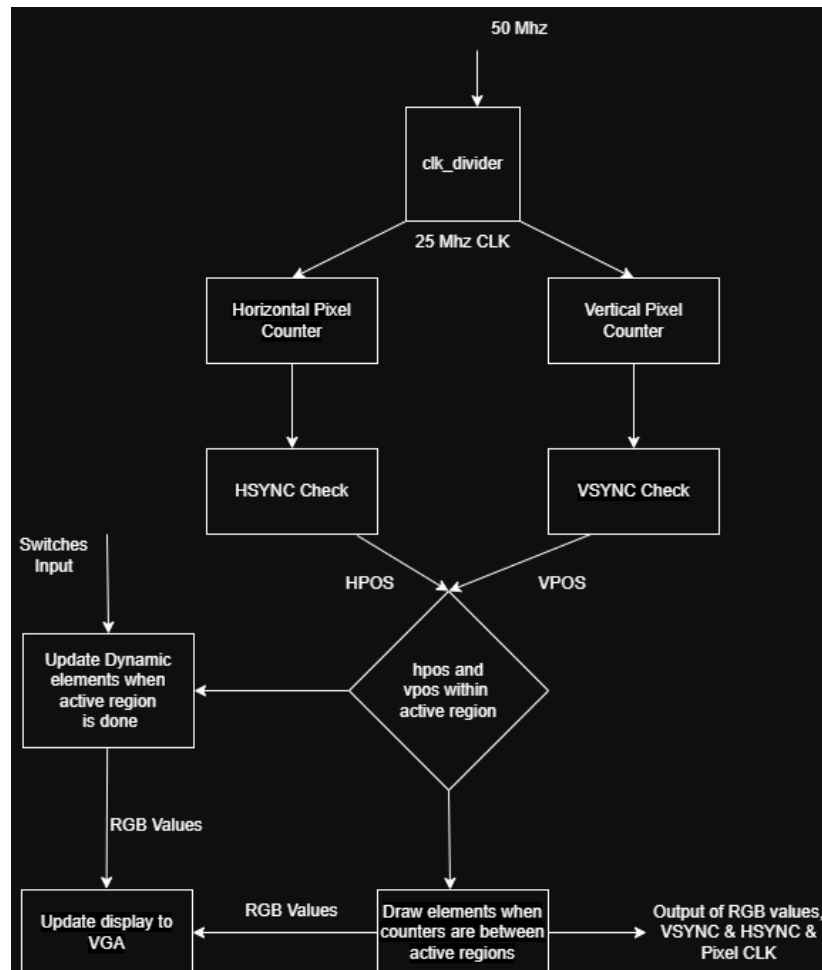


Figure 6: Process Diagram for Frame Display

The process diagram, shown in Figure 6, illustrates the operational flow of the video game processor from clock generation through final display output. The system begins with the 50 MHz onboard clock signal, which is divided by the `clk_divider` module to generate the 25 MHz pixel clock (CLK) required for VGA timing compliance. This 25 MHz clock feeds into two parallel pixel tracking modules: the Horizontal Pixel Counter and Vertical Pixel Counter, which

maintain the current pixel position (HPOS and VPOS) being rendered on the display. Each counter operates according to the VGA timing specifications, incrementing with each clock cycle and resetting after each line or frame. The HSYNC Check and VSYNC Check modules monitor the counter values and generate the appropriate synchronization signals, which transition to active-low states during the horizontal and vertical blanking intervals.

The decision block checks whether the current HPOS and VPOS values fall within the active display region (640×480 pixels). If the pixel position is within this active region, the system follows two paths: first, it updates the dynamic game elements (paddles and ball) based on switch inputs when the active region drawing is complete, ensuring smooth motion without screen tearing. Second, it determines the appropriate RGB color values for the current pixel by checking whether the position corresponds to the ball, paddles, borders, or background. The "Draw elements when counters are between active regions" block generates the RGB values based on these positional comparisons. Finally, all outputs (RGB values (8 bits per channel), VSYNC, HSYNC, and the 25 MHz Pixel CLK (DAC_CLK)) are sent to the VGA monitor for display. The "Update display to VGA" block represents the final stage where these signals interface with the Spartan-3E board's digital-to-analog converters, converting the digital RGB values into analog voltage levels for rendering on the monitor. This entire process repeats continuously at the 60 Hz refresh rate, ensuring real-time gameplay and responsive user interaction.

Results

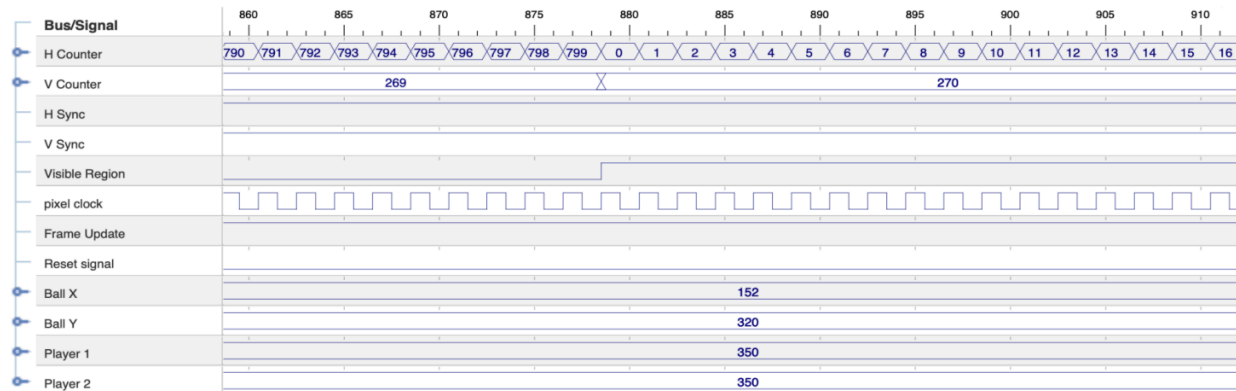


Figure 7. Waveform showing the cycle for different parameters

Figure 7 demonstrates the VGA timing signals captured during actual simulated gameplay. The horizontal counter (H Counter) increments from 790 through 799 before resetting to 0, confirming proper operation within the expected 0-799 range specified by the VGA timing parameters. The vertical counter (V Counter) transitions from line 269 to line 270 during this capture, indicating correct vertical progression. Both H Sync and V Sync signals maintain their expected states. H Sync remains high throughout this segment because the waveform captures a

portion outside the horizontal sync pulse region, while V Sync also remains high as the display is rendering within the active frame area, far from the vertical blanking interval. The Visible Region signal transitions to high precisely when the horizontal counter resets to 0, marking the beginning of the next visible row within the active 640-pixel display area. The pixel clock exhibits a consistent 25 MHz square wave pattern, providing the precise 40-nanosecond timing intervals required for proper pixel rendering. At the captured instant, the ball position registers at $X = 152$ and $Y = 320$, while both player paddles are positioned at $Y = 350$, coordinates that correspond directly to the visual elements displayed on the VGA monitor during testing.

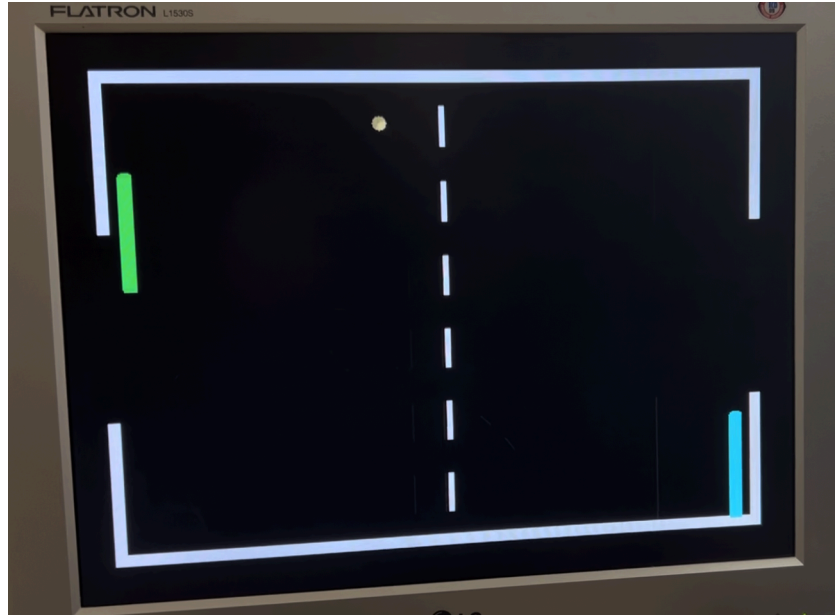


Figure 8. Ball in play during standard gameplay

Figure 8 represents a normal run of the game on the VGA monitor, where all graphics appear in the right places. The black rectangular background field along with the white rectangle border is shown to be drawn correctly. In addition, each of the two paddles on opposite sides of the screen has a solid color representation, with the left paddle shown in green and the right paddle shown in light blue. The ball is shown in mid-play moving horizontally from side to side across the screen, indicating that it's updating its horizontal position properly in response to user input. There is no indication of instability/flicker or other graphical anomalies on the display, and all of the graphics are displayed as they were supposed to be, based on shape and color. Therefore, this picture indicates that everything associated with the VGA video display (timing signals), on-screen movement, and overall performance is working as it should when the game is being played normally.

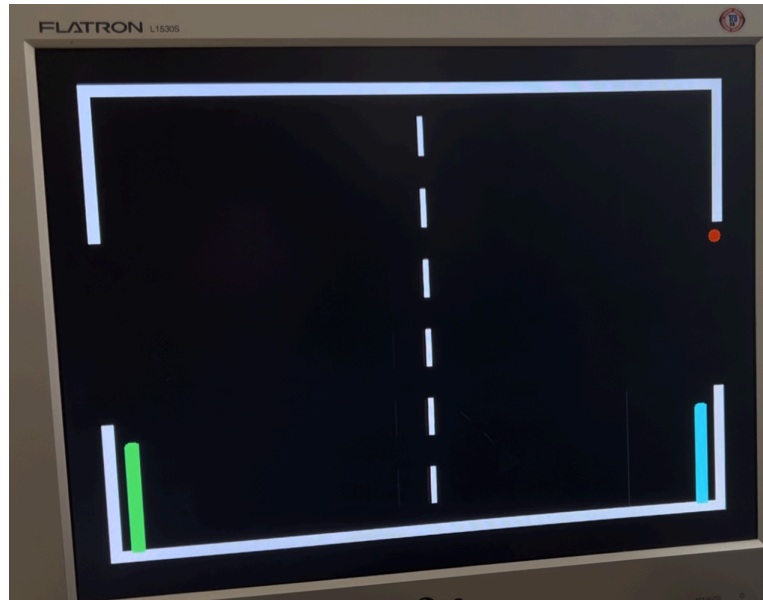


Figure 9. Scoring event captured on screen

Figure 9 shows the moment the ball enters the goal area of the playing surface, which indicates the ball crossed into the goal boundary with a successful score event. At this point in the game, the ball has been successfully switched from its regular play color to red. Therefore, the scoring detection logic and color-changing conditions have both been validated as functioning correctly. In addition, the paddles, borders, and center divider continue to be clearly displayed and stable while the VGA video signal output remains steady and doesn't show any flickering or distortion during the scoring process. This figure provides verification that the system can recognize the ball as going into the net, provide the necessary visual scoring feedback, and prepare for the next reset phase of the game based on the established rules of the game.

Conclusion

Implementation of the Simple Video Game Processor fully satisfied each of the necessary functional objectives, such as creating the proper VGA timing, creating the images for both static and dynamic objects in real time, allowing for paddle control through the use of the onboard switch controls, and implementing collision-based ball movement to behave predictably in each instance of gameplay. It was possible to achieve the desired 640×480 resolution and to drive the 60Hz refresh rate with a 25MHz pixel clock. As well, the modular nature of the VHDL design permitted separate implementation of the VGA timing, drawing, movement, and collision behavior without negatively impacting the system's performance. The completion of this project has given us hands-on experience in FPGA design, VGA interfacing, hardware debugging, and state-based control logic, and reinforced many of the concepts we have learned in class, as well as demonstrated the complete functionality of a working Pong-style game implemented on hardware.

References

- [1] L. Kirischian, "COE758 - Digital Systems Engineering Lecture Notes," Toronto Metropolitan University, Fall 2024.
- [2] L. Kirischian, "Project 2: Simple Video-Game Processor for VGA - Specification Document," COE758 Course Material, Toronto Metropolitan University, 2024.
- [3] L. Kirischian, "COE758 Design Process Tutorial," Toronto Metropolitan University, 2024.

Appendix

Source Code

vga_timing.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity vga_timing is
  Port (
    clk          : in  STD_LOGIC;
    rst          : in  STD_LOGIC;
    -- Game signals for ChipScope monitoring
    frame_tick_in : in  STD_LOGIC;
    ball_x_in    : in  INTEGER range 0 to 639;
    ball_y_in    : in  INTEGER range 0 to 479;
    p1_y_in      : in  INTEGER range 0 to 479;
    p2_y_in      : in  INTEGER range 0 to 479;
    -- VGA outputs
    pixel_clk     : out STD_LOGIC;
    hsync         : out STD_LOGIC;
    vsync         : out STD_LOGIC;
    x             : out INTEGER range 0 to 799;
    y             : out INTEGER range 0 to 524;
    visible       : out STD_LOGIC
  );
end vga_timing;

architecture Behavioral of vga_timing is
  -----
  -- VGA 640x480 @ 60 Hz timing parameters
  -----

  constant H_DISPLAY : integer := 640;
  constant H_FRONT   : integer := 16;
  constant H_SYNC     : integer := 96;
  constant H_BACK     : integer := 48;
  constant H_TOTAL    : integer := 800;

  constant V_DISPLAY : integer := 480;
  constant V_FRONT   : integer := 10;
  constant V_SYNC     : integer := 2;
  constant V_BACK     : integer := 33;

```

```

constant V_TOTAL    : integer := 525;

-----
-- Internal signals
-----

signal pix_clk      : STD_LOGIC := '0';
signal h_counter    : integer range 0 to H_TOTAL-1 := 0;
signal v_counter    : integer range 0 to V_TOTAL-1 := 0;
signal hsync_int    : STD_LOGIC := '1';
signal vsync_int    : STD_LOGIC := '1';
signal visible_int  : STD_LOGIC := '0';

-----
-- ChipScope ICON / ILA
-----

component icon
  port (CONTROL0 : inout std_logic_vector(35 downto 0));
end component;

component ila
  port (
    CONTROL : inout std_logic_vector(35 downto 0);
    CLK      : in std_logic;
    DATA    : in std_logic_vector(63 downto 0); -- EXPANDED to 64 bits
    TRIG0    : in std_logic_vector(7 downto 0)
  );
end component;

signal control0 : std_logic_vector(35 downto 0);
signal ila_data : std_logic_vector(63 downto 0); -- EXPANDED
signal trig0    : std_logic_vector(7 downto 0);

begin
-----
-- 25 MHz pixel clock divider
-----

process(clk)
begin
  if rising_edge(clk) then
    pix_clk <= not pix_clk;
  end if;
end process;

pixel_clk <= pix_clk;

-----
-- Horizontal / Vertical counters
-----

process(pix_clk, rst)
begin
  if rst = '1' then
    h_counter <= 0;
    v_counter <= 0;
  elsif rising_edge(pix_clk) then
    if h_counter = H_TOTAL - 1 then
      h_counter <= 0;
      if v_counter = V_TOTAL - 1 then
        v_counter <= 0;
      else
        v_counter <= v_counter + 1;
      end if;
    else
      h_counter <= h_counter + 1;
    end if;
  end if;
end process;

```

```

        h_counter <= h_counter + 1;
    end if;
end if;
end process;

-----
-- Sync pulses (active low)
-----
hsync_int <= '0' when (h_counter >= H_DISPLAY + H_FRONT and
                      h_counter <  H_DISPLAY + H_FRONT + H_SYNC)
                else '1';

vsync_int <= '0' when (v_counter >= V_DISPLAY + V_FRONT and
                      v_counter <  V_DISPLAY + V_FRONT + V_SYNC)
                else '1';

visible_int <= '1' when (h_counter < H_DISPLAY and v_counter < V_DISPLAY)
                    else '0';

-----
-- Outputs
-----
hsync    <= hsync_int;
vsync    <= vsync_int;
visible  <= visible_int;
x <= h_counter;
y <= v_counter;

-----
-- ENHANCED ChipScope with Game Signals
-----
icon_inst : icon
    port map (CONTROL0 => control0);

ila_inst : ila
    port map (
        CONTROL => control0,
        CLK     => clk,
        DATA   => ila_data,
        TRIG0   => trig0
    );

-- ENHANCED ILA data mapping (64 bits)
ila_data(9 downto 0)    <= std_logic_vector(to_unsigned(h_counter, 10));    -- H counter
ila_data(19 downto 10) <= std_logic_vector(to_unsigned(v_counter, 10));    -- V counter
ila_data(20) <= hsync_int;          -- HSYNC
ila_data(21) <= vsync_int;          -- VSYNC
ila_data(22) <= visible_int;        -- Visible region
ila_data(23) <= pix_clk;            -- 25MHz pixel clock
ila_data(24) <= frame_tick_in;      -- NEW: Frame update tick
ila_data(25) <= rst;                -- NEW: Reset signal

-- NEW: Game object positions for collision debugging
ila_data(35 downto 26) <= std_logic_vector(to_unsigned(ball_x_in, 10));    -- Ball X
ila_data(44 downto 36) <= std_logic_vector(to_unsigned(ball_y_in, 9));    -- Ball Y
ila_data(53 downto 45) <= std_logic_vector(to_unsigned(p1_y_in, 9));      -- Player 1 Y
ila_data(62 downto 54) <= std_logic_vector(to_unsigned(p2_y_in, 9));      -- Player 2 Y
ila_data(63) <= '0';                -- Unused

-- Trigger on VSYNC (captures one complete frame)
trig0(0) <= vsync_int;
trig0(1) <= frame_tick_in;

```

```

    trig0(7 downto 2) <= (others => '0');

end Behavioral;

```

refresh_controller.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity refresh_controller is
    Port (
        pixel_clk : in STD_LOGIC;
        rst       : in STD_LOGIC;
        frame_tick : out STD_LOGIC
    );
end refresh_controller;

architecture Behavioral of refresh_controller is
    -- Counter for frame timing: 25MHz / 416666 ≈ 60Hz
    constant FRAME_PERIOD : integer := 416666;
    signal counter : integer range 0 to FRAME_PERIOD := 0;
    signal tick_signal : STD_LOGIC := '0';
begin
    process(pixel_clk, rst)
    begin
        if rst = '1' then
            counter <= 0;
            tick_signal <= '0';
        elsif rising_edge(pixel_clk) then
            if counter >= FRAME_PERIOD - 1 then
                counter <= 0;
                tick_signal <= not tick_signal;
            else
                counter <= counter + 1;
            end if;
        end if;
    end process;

    frame_tick <= tick_signal;
end Behavioral;

```

field_renderer.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity field_renderer is
    Port (
        x_pos      : in INTEGER range 0 to 799;
        y_pos      : in INTEGER range 0 to 524;
        video_active : in STD_LOGIC;
        field_red   : out STD_LOGIC_VECTOR(7 downto 0);
        field_green : out STD_LOGIC_VECTOR(7 downto 0);
        field_blue  : out STD_LOGIC_VECTOR(7 downto 0)
    );
end field_renderer;

architecture Behavioral of field_renderer is

```

```

-----
-- Border parameters
-----

constant BORDER_INSET : integer := 30;
constant BORDER_THICK : integer := 10;

-- Calculated boundaries
constant LEFT_EDGE    : integer := BORDER_INSET;
constant RIGHT_EDGE   : integer := 640 - BORDER_INSET;
constant TOP_EDGE     : integer := BORDER_INSET;
constant BOTTOM_EDGE  : integer := 480 - BORDER_INSET;

constant LEFT_BORDER_END    : integer := LEFT_EDGE + BORDER_THICK;
constant RIGHT_BORDER_START : integer := RIGHT_EDGE - BORDER_THICK;
constant TOP_BORDER_END     : integer := TOP_EDGE + BORDER_THICK;
constant BOTTOM_BORDER_START : integer := BOTTOM_EDGE - BORDER_THICK;

-- Goal opening parameters
constant GOAL_TOP    : integer := 165;
constant GOAL_BOTTOM : integer := 325;

-----
-- CENTER LINE parameters (perfectly calculated)
-----

constant MID_X1 : integer := 318;
constant MID_X2 : integer := 322;

-- Playing field height (excluding borders)
constant FIELD_HEIGHT : integer := BOTTOM_BORDER_START - TOP_BORDER_END; -- 400 pixels

-- Dash parameters for perfect symmetry
constant DASH_LENGTH : integer := 35; -- Each dash is 35 pixels
constant DASH_GAP    : integer := 30; -- Gap between dashes is 30 pixels
constant DASH_PERIOD : integer := DASH_LENGTH + DASH_GAP; -- 65 pixels per cycle

-- Number of complete dashes that fit: 400 / 65 = 6.15
-- So we use 6 dashes with equal spacing
constant NUM_DASHES : integer := 6;

-- Calculate starting position for perfect centering
-- Total space needed: 6 dashes (210px) + 5 gaps (150px) = 360px
-- Remaining space: 400 - 360 = 40px (20px margin top and bottom)
constant DASH_START_Y : integer := TOP_BORDER_END + 20;

begin
  process(x_pos, y_pos, video_active)
    variable is_white_border : boolean;
    variable dash_index : integer;
    variable dash_y_start : integer;
    variable dash_y_end : integer;
  begin
    field_red   <= (others => '0');
    field_green <= (others => '0');
    field_blue  <= (others => '0');

    if video_active = '1' then
      -----
      -- Default: Green playing field
      -----
      field_red   <= (others => '0');
      field_green <= (others => '0');
      field_blue  <= (others => '0');
    end if;
  end
end

```

```

is_white_border := false;

-----
-- TOP BORDER (complete, including corners)
-----
if (y_pos >= TOP_EDGE and y_pos < TOP_BORDER_END) and
   (x_pos >= LEFT_EDGE and x_pos < RIGHT_EDGE) then
   is_white_border := true;
end if;

-----
-- BOTTOM BORDER (complete, including corners)
-----
if (y_pos >= BOTTOM_BORDER_START and y_pos < BOTTOM_EDGE) and
   (x_pos >= LEFT_EDGE and x_pos < RIGHT_EDGE) then
   is_white_border := true;
end if;

-----
-- LEFT BORDER (excluding goal opening)
-----
if (x_pos >= LEFT_EDGE and x_pos < LEFT_BORDER_END) and
   (y_pos >= TOP_BORDER_END and y_pos < BOTTOM_BORDER_START) then
   if (y_pos < GOAL_TOP or y_pos >= GOAL_BOTTOM) then
     is_white_border := true;
   end if;
end if;

-----
-- RIGHT BORDER (excluding goal opening)
-----
if (x_pos >= RIGHT_BORDER_START and x_pos < RIGHT_EDGE) and
   (y_pos >= TOP_BORDER_END and y_pos < BOTTOM_BORDER_START) then
   if (y_pos < GOAL_TOP or y_pos >= GOAL_BOTTOM) then
     is_white_border := true;
   end if;
end if;

-----
-- Apply white border color
-----
if is_white_border then
   field_red    <= (others => '1');
   field_green  <= (others => '1');
   field_blue   <= (others => '1');
end if;

-----
-- CENTER DASHED LINE (black, symmetric, equal-length dashes)
-----
if (x_pos >= MID_X1 and x_pos <= MID_X2) then
   -- Check each of the 6 dashes
   for i in 0 to NUM_DASHES-1 loop
      dash_y_start := DASH_START_Y + (i * DASH_PERIOD);
      dash_y_end   := dash_y_start + DASH_LENGTH;

      if (y_pos >= dash_y_start and y_pos < dash_y_end) then
         -- Draw black dash
         field_red    <= (others => '1');
         field_green  <= (others => '1');
         field_blue   <= (others => '1');
      end if;
   end loop;
end if;

```

```

        exit; -- No need to check other dashes
    end if;
end loop;
end if;
end if;
end process;
end Behavioral;

```

player_movement.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity player_movement is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        frame_tick : in STD_LOGIC;
        move_up : in STD_LOGIC;
        move_down : in STD_LOGIC;
        player_side : in STD_LOGIC; -- '1' for left, '0' for right
        paddle_y_pos : out INTEGER range 0 to 479;
        x_pos : in INTEGER range 0 to 799;
        y_pos : in INTEGER range 0 to 524;
        video_active : in STD_LOGIC;
        player_red : out STD_LOGIC_VECTOR(7 downto 0);
        player_green : out STD_LOGIC_VECTOR(7 downto 0);
        player_blue : out STD_LOGIC_VECTOR(7 downto 0)
    );
end player_movement;

architecture Behavioral of player_movement is
    -----
    -- Paddle dimensions
    -----
    constant PADDLE_WIDTH : integer := 16;
    constant PADDLE_HEIGHT : integer := 90;
    constant PADDLE_SPEED : integer := 4;

    -----
    -- Border parameters (must match field_renderer)
    -----
    constant BORDER_INSET : integer := 30;
    constant BORDER_THICK : integer := 10;

    -----
    -- Calculated positions for perfect alignment
    -----
    -- Paddle X positions: Just inside the border
    constant LEFT_X : integer := BORDER_INSET + BORDER_THICK + 8; -- 48 pixels from left edge
    constant RIGHT_X : integer := 640 - BORDER_INSET - BORDER_THICK - PADDLE_WIDTH - 8; --
Symmetric

    -- Movement limits: Stay within borders
    constant TOP_LIMIT : integer := BORDER_INSET + BORDER_THICK; -- 40
    constant BOTTOM_LIMIT : integer := 480 - BORDER_INSET - BORDER_THICK - PADDLE_HEIGHT; --
350

    -----
    -- Paddle state

```

```

-----
signal paddle_y : integer range 0 to 479 := 210;  -- Centered vertically
signal prev_tick : STD_LOGIC := '0';

begin
-----
-- Paddle movement control (edge-triggered on frame_tick)
-----
process(clk, rst)
begin
    if rst = '1' then
        paddle_y <= 210;  -- Center position
        prev_tick <= '0';
    elsif rising_edge(clk) then
        -- Detect rising edge of frame_tick
        if frame_tick = '1' and prev_tick = '0' then
            if move_up = '1' and paddle_y > TOP_LIMIT then
                paddle_y <= paddle_y - PADDLE_SPEED;
            elsif move_down = '1' and paddle_y < BOTTOM_LIMIT then
                paddle_y <= paddle_y + PADDLE_SPEED;
            end if;
        end if;
        prev_tick <= frame_tick;
    end if;
end process;

paddle_y_pos <= paddle_y;

-----
-- Paddle rendering with rounded corners (optional polish)
-----
process(x_pos, y_pos, video_active, paddle_y, player_side)
    variable x_start, x_end : integer;
    variable y_start, y_end : integer;
    variable dx, dy : integer;
begin
    player_red    <= (others => '0');
    player_green  <= (others => '0');
    player_blue   <= (others => '0');

    if video_active = '1' then
        -- Determine paddle X position
        if player_side = '1' then
            x_start := LEFT_X;
        else
            x_start := RIGHT_X;
        end if;
        x_end := x_start + PADDLE_WIDTH;
        y_start := paddle_y;
        y_end := paddle_y + PADDLE_HEIGHT;

        -- Check if current pixel is within paddle bounds
        if (x_pos >= x_start and x_pos < x_end and
            y_pos >= y_start and y_pos < y_end) then

            -- Optional: Rounded corners (comment out for square paddles)
            dx := 0;
            dy := 0;

            -- Top-left corner
            if (x_pos < x_start + 3 and y_pos < y_start + 3) then
                dx := x_start + 3 - x_pos;

```

```

dy := y_start + 3 - y_pos;
if (dx*dx + dy*dy > 9) then
  -- Skip this pixel (outside rounded corner)
else
  -- Draw paddle color
  if player_side = '1' then
    -- Left paddle: Blue
    player_green <= (others => '1');
  else
    -- Right paddle: Pink (Green + Blue)
    player_green <= (others => '1');
    player_blue <= (others => '1');
  end if;
end if;
-- Top-right corner
elsif (x_pos >= x_end - 3 and y_pos < y_start + 3) then
  dx := x_pos - (x_end - 3);
  dy := y_start + 3 - y_pos;
  if (dx*dx + dy*dy > 9) then
    -- Skip
  else
    if player_side = '1' then
      player_green <= (others => '1');
    else
      player_green <= (others => '1');
      player_blue <= (others => '1');
    end if;
  end if;
-- Bottom-left corner
elsif (x_pos < x_start + 3 and y_pos >= y_end - 3) then
  dx := x_start + 3 - x_pos;
  dy := y_pos - (y_end - 3);
  if (dx*dx + dy*dy > 9) then
    -- Skip
  else
    if player_side = '1' then
      player_green <= (others => '1');
    else
      player_green <= (others => '1');
      player_blue <= (others => '1');
    end if;
  end if;
-- Bottom-right corner
elsif (x_pos >= x_end - 3 and y_pos >= y_end - 3) then
  dx := x_pos - (x_end - 3);
  dy := y_pos - (y_end - 3);
  if (dx*dx + dy*dy > 9) then
    -- Skip
  else
    if player_side = '1' then
      player_green <= (others => '1');
    else
      player_green <= (others => '1');
      player_blue <= (others => '1');
    end if;
  end if;
else
  -- Normal paddle area (not corners)
  if player_side = '1' then
    player_green <= (others => '1');
  else
    player_green <= (others => '1');
  end if;
end if;

```

```

        player_blue <= (others => '1');
    end if;
end if;
end if;
end if;
end process;
end Behavioral;

```

ball_physics.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ball_physics is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;
        frame_tick : in STD_LOGIC;
        player1_y : in INTEGER range 0 to 479;
        player2_y : in INTEGER range 0 to 479;
        x_pos : in INTEGER range 0 to 799;
        y_pos : in INTEGER range 0 to 524;
        video_active : in STD_LOGIC;
        -- ChipScope outputs (for monitoring)
        ball_x_out : out INTEGER range 0 to 639;
        ball_y_out : out INTEGER range 0 to 479;
        -- VGA outputs
        ball_red : out STD_LOGIC_VECTOR(7 downto 0);
        ball_green : out STD_LOGIC_VECTOR(7 downto 0);
        ball_blue : out STD_LOGIC_VECTOR(7 downto 0)
    );
end ball_physics;

architecture Behavioral of ball_physics is
    -----
    -- Ball parameters
    -----
    constant BALL_SIZE      : integer := 12;      -- Ball diameter
    constant BALL_SPEED     : integer := 4;       -- Pixels per frame

    -----
    -- Border parameters (must match field_renderer)
    -----
    constant BORDER_INSET : integer := 30;
    constant BORDER_THICK : integer := 10;

    constant TOP_WALL      : integer := BORDER_INSET + BORDER_THICK;      -- 40
    constant BOTTOM_WALL   : integer := 480 - BORDER_INSET - BORDER_THICK; -- 440

    -----
    -- Paddle parameters (must match player_movement)
    -----
    constant PADDLE_WIDTH  : integer := 16;
    constant PADDLE_HEIGHT : integer := 90;

    -- Paddle X positions (must match player_movement exactly)
    constant LEFT_PADDLE_X : integer := BORDER_INSET + BORDER_THICK + 8;      -- 48
    constant RIGHT_PADDLE_X : integer := 640 - BORDER_INSET - BORDER_THICK - PADDLE_WIDTH - 8;
    -- 576

```

```

-----
-- Goal parameters
-----
constant GOAL_TOP      : integer := 150;
constant GOAL_BOTTOM   : integer := 350;
constant LEFT_GOAL_X   : integer := BORDER_INSET + BORDER_THICK;      -- 40
constant RIGHT_GOAL_X  : integer := 640 - BORDER_INSET - BORDER_THICK; -- 600

-----
-- Ball state variables
-----
signal ball_x : integer range 0 to 639 := 320;
signal ball_y : integer range 0 to 479 := 240;
signal velocity_x : integer := 1;    -- 1 = right, -1 = left
signal velocity_y : integer := 1;    -- 1 = down, -1 = up
signal goal_scored : STD_LOGIC := '0';
signal respawn_counter : integer range 0 to 100 := 0;
signal prev_tick : STD_LOGIC := '0';

begin
-----
-- Ball movement and collision detection
-----
process(clk, rst)
begin
    if rst = '1' then
        ball_x <= 320;
        ball_y <= 240;
        velocity_x <= 1;
        velocity_y <= 1;
        goal_scored <= '0';
        respawn_counter <= 0;
        prev_tick <= '0';

    elsif rising_edge(clk) then
        -- Only update on rising edge of frame_tick
        if frame_tick = '1' and prev_tick = '0' then

            if goal_scored = '0' then
                -----
                -- MOVE THE BALL
                -----
                ball_x <= ball_x + (velocity_x * BALL_SPEED);
                ball_y <= ball_y + (velocity_y * BALL_SPEED);

                -----
                -- TOP/BOTTOM WALL COLLISION
                -----
                if ball_y <= TOP_WALL then
                    velocity_y <= 1;    -- Bounce down
                elsif ball_y + BALL_SIZE >= BOTTOM_WALL then
                    velocity_y <= -1;   -- Bounce up
                end if;

                -----
                -- SIDE WALL COLLISION (above/below goal areas)
                -- Ball bounces off white borders outside goal openings
                -----
                if velocity_x < 0 then
                    -- Moving left: check left wall
                    if (ball_x <= LEFT_GOAL_X) and
                       (ball_y <= GOAL_TOP or ball_y + BALL_SIZE >= GOAL_BOTTOM) then

```

```

        velocity_x <= 1; -- Bounce right
    end if;
else
    -- Moving right: check right wall
    if (ball_x + BALL_SIZE >= RIGHT_GOAL_X) and
        (ball_y <= GOAL_TOP or ball_y + BALL_SIZE >= GOAL_BOTTOM) then
        velocity_x <= -1; -- Bounce left
    end if;
end if;

-----
-- LEFT PADDLE COLLISION (edge detection)
-----
if velocity_x < 0 then
    -- Only check when ball is moving left
    if (ball_x <= LEFT_PADDLE_X + PADDLE_WIDTH) and
        (ball_x > LEFT_PADDLE_X + PADDLE_WIDTH - BALL_SPEED - 2) and
        (ball_y + BALL_SIZE >= player1_y) and
        (ball_y <= player1_y + PADDLE_HEIGHT) then
        velocity_x <= 1; -- Bounce right
    end if;
end if;

-----
-- RIGHT PADDLE COLLISION (edge detection)
-----
if velocity_x > 0 then
    -- Only check when ball is moving right
    if (ball_x + BALL_SIZE >= RIGHT_PADDLE_X) and
        (ball_x + BALL_SIZE < RIGHT_PADDLE_X + BALL_SPEED + 2) and
        (ball_y + BALL_SIZE >= player2_y) and
        (ball_y <= player2_y + PADDLE_HEIGHT) then
        velocity_x <= -1; -- Bounce left
    end if;
end if;

-----
-- GOAL DETECTION
-- Ball enters goal area (between Y=150 and Y=350)
-----
if (ball_x < LEFT_GOAL_X and
    ball_y > GOAL_TOP and
    ball_y < GOAL_BOTTOM) or
    (ball_x + BALL_SIZE > RIGHT_GOAL_X and
    ball_y > GOAL_TOP and
    ball_y < GOAL_BOTTOM) then
    goal_scored <= '1';
    respawn_counter <= 0;
end if;

else
    -----
    -- RESPAWN DELAY after scoring
    -----
    if respawn_counter < 70 then
        respawn_counter <= respawn_counter + 1;
    else
        -- Respawn at center
        ball_x <= 320;
        ball_y <= 240;
        -- Reverse direction for fairness
        velocity_x <= -velocity_x;
    end if;
end if;

```

```

        goal_scored <= '0';
    end if;
end if;
end if;

    prev_tick <= frame_tick;
end if;
end process;

-----
-- ChipScope outputs (for debugging)
-----

ball_x_out <= ball_x;
ball_y_out <= ball_y;

-----
-- Ball rendering (round shape using distance calculation)
-----

process(x_pos, y_pos, video_active, ball_x, ball_y, goal_scored)
    variable dx, dy, dist2 : integer;
    variable radius_sq : integer;
begin
    ball_red    <= "00000000";
    ball_green  <= "00000000";
    ball_blue   <= "00000000";

    if video_active = '1' then
        -- Calculate distance from pixel to ball center
        dx := x_pos - (ball_x + BALL_SIZE/2);
        dy := y_pos - (ball_y + BALL_SIZE/2);
        dist2 := dx*dx + dy*dy;
        radius_sq := (BALL_SIZE/2) * (BALL_SIZE/2);

        -- Draw circular ball if within radius
        if dist2 <= radius_sq then
            if goal_scored = '0' then
                -- Yellow ball during normal play
                ball_red    <= "11111101";
                ball_green  <= "11101111";
                ball_blue   <= "01111100";
            else
                -- Red ball when scored
                ball_red    <= "11111111";
                ball_green  <= "00000000";
                ball_blue   <= "00000000";
            end if;
        end if;
    end if;
end if;
end process;

end Behavioral;

```

pong_top.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity pong_top is
    Port (
        clk : in STD_LOGIC;
        rst : in STD_LOGIC;

```

```

    SW0 : in STD_LOGIC;
    SW1 : in STD_LOGIC;
    SW2 : in STD_LOGIC;
    SW3 : in STD_LOGIC;
    H : out STD_LOGIC;
    V : out STD_LOGIC;
    DAC_CLK : out STD_LOGIC;
    Rout : out STD_LOGIC_VECTOR(7 downto 0);
    Gout : out STD_LOGIC_VECTOR(7 downto 0);
    Bout : out STD_LOGIC_VECTOR(7 downto 0)
);
end pong_top;

architecture Structural of pong_top is
-----
-- Component declarations
-----

component vga_timing
  Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    frame_tick_in : in STD_LOGIC;
    ball_x_in : in INTEGER range 0 to 639;
    ball_y_in : in INTEGER range 0 to 479;
    p1_y_in : in INTEGER range 0 to 479;
    p2_y_in : in INTEGER range 0 to 479;
    pixel_clk : out STD_LOGIC;
    hsync : out STD_LOGIC;
    vsync : out STD_LOGIC;
    x : out INTEGER range 0 to 799;
    y : out INTEGER range 0 to 524;
    visible : out STD_LOGIC
  );
end component;

component refresh_controller
  Port (
    pixel_clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    frame_tick : out STD_LOGIC
  );
end component;

component field_renderer
  Port (
    x_pos : in INTEGER range 0 to 799;
    y_pos : in INTEGER range 0 to 524;
    video_active : in STD_LOGIC;
    field_red : out STD_LOGIC_VECTOR(7 downto 0);
    field_green : out STD_LOGIC_VECTOR(7 downto 0);
    field_blue : out STD_LOGIC_VECTOR(7 downto 0)
  );
end component;

component player_movement
  Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    frame_tick : in STD_LOGIC;
    move_up : in STD_LOGIC;
    move_down : in STD_LOGIC;
    player_side : in STD_LOGIC;

```

```

    paddle_y_pos : out INTEGER range 0 to 479;
    x_pos : in INTEGER range 0 to 799;
    y_pos : in INTEGER range 0 to 524;
    video_active : in STD_LOGIC;
    player_red : out STD_LOGIC_VECTOR(7 downto 0);
    player_green : out STD_LOGIC_VECTOR(7 downto 0);
    player_blue : out STD_LOGIC_VECTOR(7 downto 0)
  );
end component;

component ball_physics
  Port (
    clk : in STD_LOGIC;
    rst : in STD_LOGIC;
    frame_tick : in STD_LOGIC;
    player1_y : in INTEGER range 0 to 479;
    player2_y : in INTEGER range 0 to 479;
    x_pos : in INTEGER range 0 to 799;
    y_pos : in INTEGER range 0 to 524;
    video_active : in STD_LOGIC;
    ball_x_out : out INTEGER range 0 to 639; -- NEW: for ChipScope
    ball_y_out : out INTEGER range 0 to 479; -- NEW: for ChipScope
    ball_red : out STD_LOGIC_VECTOR(7 downto 0);
    ball_green : out STD_LOGIC_VECTOR(7 downto 0);
    ball_blue : out STD_LOGIC_VECTOR(7 downto 0)
  );
end component;

-----
-- Internal signals
-----

signal pxclk : STD_LOGIC;
signal video_on : STD_LOGIC;
signal tick : STD_LOGIC;
signal x_coord : INTEGER range 0 to 799;
signal y_coord : INTEGER range 0 to 524;

signal p1_red, p1_green, p1_blue : STD_LOGIC_VECTOR(7 downto 0);
signal p2_red, p2_green, p2_blue : STD_LOGIC_VECTOR(7 downto 0);
signal b_red, b_green, b_blue : STD_LOGIC_VECTOR(7 downto 0);
signal f_red, f_green, f_blue : STD_LOGIC_VECTOR(7 downto 0);

signal p1_y_position : INTEGER range 0 to 479;
signal p2_y_position : INTEGER range 0 to 479;

-- NEW: Signals for ChipScope monitoring
signal ball_x_pos : INTEGER range 0 to 639;
signal ball_y_pos : INTEGER range 0 to 479;

begin
  -----
  -- VGA timing generator (with ChipScope)
  -----
  timing_gen : vga_timing
    port map (
      clk => clk,
      rst => rst,
      frame_tick_in => tick,
      ball_x_in => ball_x_pos,
      ball_y_in => ball_y_pos,
      p1_y_in => p1_y_position,
      p2_y_in => p2_y_position,

```

```

    pixel_clk => pxclk,
    hsync => H,
    vsync => V,
    x => x_coord,
    y => y_coord,
    visible => video_on
);

DAC_CLK <= pxclk;

-----
-- Refresh rate controller
-----
refresh_ctrl : refresh_controller
  port map (
    pixel_clk => pxclk,
    rst => rst,
    frame_tick => tick
  );

-----
-- Field renderer
-----
field_gen : field_renderer
  port map (
    x_pos => x_coord,
    y_pos => y_coord,
    video_active => video_on,
    field_red => f_red,
    field_green => f_green,
    field_blue => f_blue
  );

-----
-- Player 1 (left, blue)
-----
player1_ctrl : player_movement
  port map (
    clk => clk,
    rst => rst,
    frame_tick => tick,
    move_up => SW0,
    move_down => SW1,
    player_side => '1',
    paddle_y_pos => p1_y_position,
    x_pos => x_coord,
    y_pos => y_coord,
    video_active => video_on,
    player_red => p1_red,
    player_green => p1_green,
    player_blue => p1_blue
  );

-----
-- Player 2 (right, pink)
-----
player2_ctrl : player_movement
  port map (
    clk => clk,
    rst => rst,
    frame_tick => tick,
    move_up => SW2,

```

```

    move_down => SW3,
    player_side => '0',
    paddle_y_pos => p2_y_position,
    x_pos => x_coord,
    y_pos => y_coord,
    video_active => video_on,
    player_red => p2_red,
    player_green => p2_green,
    player_blue => p2_blue
);

-----
-- Ball controller
-----
ball_ctrl : ball_physics
port map (
    clk => clk,
    rst => rst,
    frame_tick => tick,
    player1_y => p1_y_position,
    player2_y => p2_y_position,
    x_pos => x_coord,
    y_pos => y_coord,
    video_active => video_on,
    ball_x_out => ball_x_pos,
    ball_y_out => ball_y_pos,
    ball_red => b_red,
    ball_green => b_green,
    ball_blue => b_blue
);

-----
-- Color priority multiplexer (Ball > Players > Field)
-----
process(f_red, f_green, f_blue, p1_red, p1_green, p1_blue,
        p2_red, p2_green, p2_blue, b_red, b_green, b_blue)
begin
    -- Default to field
    Rout <= f_red;
    Gout <= f_green;
    Bout <= f_blue;

    -- Player 1 overrides field
    if p1_red /= "00000000" or p1_green /= "00000000" or p1_blue /= "00000000" then
        Rout <= p1_red;
        Gout <= p1_green;
        Bout <= p1_blue;
    end if;

    -- Player 2 overrides player 1 and field
    if p2_red /= "00000000" or p2_green /= "00000000" or p2_blue /= "00000000" then
        Rout <= p2_red;
        Gout <= p2_green;
        Bout <= p2_blue;
    end if;

    -- Ball has highest priority
    if b_red /= "00000000" or b_green /= "00000000" or b_blue /= "00000000" then
        Rout <= b_red;
        Gout <= b_green;
        Bout <= b_blue;
    end if;
end process;

```

```

    end process;
end Structural;

```

pong.ucf

```

# =====
# System Clock (50 MHz)
# =====
NET "clk" LOC = "C9";
NET "clk" TNM_NET = "clk";
TIMESPEC TS_clk = PERIOD "clk" 20 ns HIGH 50%;

# =====
# Reset Button
# =====
NET "rst" LOC = "B18";

# =====
# VGA Interface
# =====
# Horizontal Sync
NET "H" LOC = "C5";

# Vertical Sync
NET "V" LOC = "D5";

# Pixel Clock (25 MHz output to DAC)
NET "DAC_CLK" LOC = "A4";

# =====
# VGA Color Channels (8 bits each)
# =====
# Red Channel
NET "Rout<7>" LOC = "E8";
NET "Rout<6>" LOC = "F8";
NET "Rout<5>" LOC = "C7";
NET "Rout<4>" LOC = "D7";
NET "Rout<3>" LOC = "F7";
NET "Rout<2>" LOC = "E7";
NET "Rout<1>" LOC = "B6";
NET "Rout<0>" LOC = "A6";

# Green Channel
NET "Gout<7>" LOC = "F12";
NET "Gout<6>" LOC = "E12";
NET "Gout<5>" LOC = "E11";
NET "Gout<4>" LOC = "F11";
NET "Gout<3>" LOC = "C11";
NET "Gout<2>" LOC = "D11";
NET "Gout<1>" LOC = "E9";
NET "Gout<0>" LOC = "F9";

# Blue Channel
NET "Bout<7>" LOC = "B16";
NET "Bout<6>" LOC = "A16";
NET "Bout<5>" LOC = "D14";
NET "Bout<4>" LOC = "C14";
NET "Bout<3>" LOC = "B14";
NET "Bout<2>" LOC = "A14";
NET "Bout<1>" LOC = "B13";

```

```

NET "Bout<0>" LOC = "A13";

# =====
# Player Control Switches
# =====
# Player 1 Controls (Left Paddle - Blue)
NET "SW0" LOC = "N17"; # Player 1 Up
NET "SW1" LOC = "H18"; # Player 1 Down

# Player 2 Controls (Right Paddle - Pink)
NET "SW2" LOC = "L14"; # Player 2 Up
NET "SW3" LOC = "L13"; # Player 2 Down

#####
# End of Constraints
#####

```